

ETIKA & KEAMANAN PERANGKAT LUNAK

Studi Literatur, Diskusi AI, Studi Kasus & Kerangka Etika Pengembangan

Privasi | Keamanan | Bias Algoritma | Kerangka Etika

BAGIAN I: STUDI LITERATUR

1.1 Privasi & Keamanan Tanpa Mengorbankan Fungsionalitas

Salah satu tantangan terbesar dalam pengembangan perangkat lunak modern adalah menyeimbangkan privasi dan keamanan dengan tetap mempertahankan fungsionalitas yang dibutuhkan pengguna. Keduanya sering dianggap bertentangan, padahal dengan pendekatan yang tepat keduanya dapat berjalan beriringan.

Prinsip Privacy by Design

Privacy by Design (PbD) adalah pendekatan yang mengintegrasikan perlindungan privasi sejak tahap perencanaan, bukan sebagai tambahan setelah sistem selesai dibangun. Terdapat tujuh prinsip utama:

- Proaktif, bukan reaktif — mencegah masalah privasi sebelum terjadi
- Privasi sebagai pengaturan default
- Privasi tertanam dalam desain sistem
- Fungsionalitas penuh — positive-sum, bukan zero-sum
- Keamanan end-to-end sepanjang siklus data
- Visibilitas dan transparansi kepada pengguna
- Menghormati privasi pengguna

Strategi Teknis Menyeimbangkan Keamanan dan Fungsionalitas

Berikut adalah strategi yang dapat diterapkan oleh programmer:

Strategi	Implementasi & Dampak
Data Minimization	Hanya kumpulan data yang benar-benar dibutuhkan. Kurangi permukaan serangan tanpa mengurangi fitur inti.
Enkripsi End-to-End	Gunakan TLS 1.3 untuk transit, AES-256 untuk data at rest. Pengguna tetap bisa berinteraksi normal.
Autentikasi Multi-Faktor	MFA meningkatkan keamanan signifikan tanpa menghilangkan akses pengguna yang sah.
Role-Based Access Control	Hak akses sesuai peran — pengguna hanya melihat data yang relevan, sistem tetap fungsional.
Tokenisasi & Pseudonimisasi	Data sensitif diganti token; analitik dan fitur lain tetap berjalan menggunakan token.
Differential Privacy	Teknik statistik menambahkan noise ke data agregat sehingga privasi terlindungi, analisis tetap valid.
Sandboxing & Least Privilege	Komponen berjalan dengan hak minimal; kerusakan akibat bug atau serangan terisolasi.

Kesimpulan Kunci

Privasi dan fungsionalitas bukan pilihan biner. Dengan menerapkan Privacy by Design sejak awal, programmer dapat membangun sistem yang aman, patuh regulasi (seperti GDPR/UU PDP Indonesia), dan tetap memberikan pengalaman pengguna yang baik.

1.2 Bias dalam Algoritma: Diskusi dengan AI

Bias algoritma terjadi ketika sistem menghasilkan output yang secara sistematis tidak adil terhadap kelompok tertentu, baik karena data latih, desain model, maupun konteks penggunaannya.

Contoh Nyata Bias Algoritma

Kasus 1: COMPAS — Sistem Penilaian Risiko Kriminal (AS)

COMPAS (Correctional Offender Management Profiling for Alternative Sanctions) adalah sistem yang digunakan pengadilan AS untuk memprediksi kemungkinan seseorang melakukan kejahatan ulang. Investigasi ProPublica (2016) menemukan:

- Orang kulit hitam dua kali lebih sering diklasifikasikan sebagai risiko tinggi secara keliru (false positive)
- Orang kulit putih lebih sering diklasifikasikan risiko rendah secara keliru (false negative)
- Algoritma menggunakan fitur seperti kode pos yang berkorelasi dengan ras akibat segregasi historis

Pelajaran:

Data historis yang mencerminkan ketidakadilan masa lalu akan mewariskan ketidakadilan itu ke dalam keputusan otomatis.

Kasus 2: Algoritma Rekrutmen Amazon

Amazon mengembangkan sistem AI untuk menyaring CV pelamar kerja antara 2014-2018. Sistem ini akhirnya dihentikan karena:

- Dilatih dengan data historis rekrutmen selama 10 tahun yang didominasi pelamar pria
- Algoritma belajar menghukum kata-kata seperti "women's" (misal: "women's chess club")
- Model menilai lebih rendah lulusan universitas yang secara historis banyak perempuan
- Bias muncul meskipun variabel gender dihapus secara eksplisit dari input

Kasus 3: Google Photos — Pengenalan Wajah

Pada 2015, Google Photos secara salah mengklasifikasikan foto orang kulit hitam sebagai gorila. Akar masalahnya adalah dataset pelatihan yang sangat tidak seimbang, dengan representasi wajah kulit putih yang jauh lebih dominan.

Apakah Bias Dapat Dihilangkan Sepenuhnya?

Jawaban singkat: Tidak sepenuhnya. Tetapi bias dapat dikelola dan diminimalkan secara signifikan.

Sumber Bias	Strategi Mitigasi
Data historis tidak representatif	Augmentasi data, oversampling, pengumpulan data baru yang inklusif
Pemilihan fitur berkorelasi proxy	Analisis korelasi fitur, penghapusan proxy ras/gender
Definisi 'keadilan' yang ambigu	Tetapkan metrik fairness yang eksplisit (demographic parity, equalized odds)
Kurangnya perspektif tim developer	Tim pengembang yang beragam (gender, etnis, latar belakang)
Tidak ada audit pasca-deploy	Monitoring berkelanjutan, audit berkala oleh pihak independen

Perspektif Kritis

Bahkan definisi 'adil' pun bisa bias: Fairness through Awareness vs Fairness through Unawareness adalah dua filosofi yang saling bertentangan. Komunitas ML belum memiliki konsensus tunggal tentang definisi bias yang universal — ini adalah masalah etika, bukan hanya teknis.

BAGIAN II: STUDI KASUS — PELANGGARAN KEAMANAN

2.1 Analisis Pelanggaran Keamanan pada Aplikasi Populer

Berikut adalah analisis tiga pelanggaran keamanan besar beserta solusi yang dapat diterapkan programmer.

Kasus A: Kebocoran Data Facebook — Cambridge Analytica (2018)

Skala: Data 87 juta pengguna disalahgunakan untuk profiling politik.

Aspek	Detail
Vektor Serangan	Aplikasi kuis pihak ketiga mengeksploitasi API Facebook yang memberikan akses data teman pengguna tanpa consent eksplisit
Kerentanan Utama	Kurangnya pembatasan akses API, tidak ada consent granular, minimnya audit aplikasi pihak ketiga
Dampak	87 juta profil pengguna, denda GDPR, kerusakan reputasi masif, regulasi baru

Solusi yang Harus Diterapkan Programmer:

1. Implementasi OAuth 2.0 dengan scope yang granular — pengguna memilih data spesifik yang dibagikan
2. Rate limiting ketat pada API endpoint yang mengakses data sensitif
3. Audit log setiap akses data oleh pihak ketiga
4. Data expiration — token API pihak ketiga kadaluarsa setelah periode tertentu
5. Penetration testing API secara berkala
6. Zero-trust architecture: verifikasi identitas di setiap request

Kasus B: Kebocoran Data Tokopedia (2020)

Skala: 91 juta akun pengguna, termasuk hash password, nomor HP, dan email.

Aspek	Detail
Vektor Serangan	Diduga SQL Injection atau kompromi credential internal yang memberikan akses ke database produksi
Kerentanan Utama	Database tidak terenkripsi penuh, tidak ada deteksi anomali akses, password di-hash dengan algoritma lemah (SHA-1/Md5)
Dampak	91 juta akun dijual di forum dark web, kepercayaan pengguna turun

Solusi yang Harus Diterapkan Programmer:

7. Gunakan bcrypt, Argon2, atau scrypt untuk hashing password — JANGAN MD5/SHA-1
8. Enkripsi database end-to-end dengan key management yang proper (AWS KMS, HashiCorp Vault)
9. Prepared statements dan parameterized queries untuk mencegah SQL Injection
10. Network segmentation — database tidak boleh langsung terekspos ke internet
11. Intrusion Detection System (IDS) dan anomaly detection pada query database
12. Principle of least privilege untuk akun database

Kasus C: Log4Shell — Kerentanan Log4j (2021)

Skala: Kerentanan di library logging Java yang digunakan miliaran sistem. CVSS Score 10/10.

Aspek	Detail
Vektor Serangan	JNDI injection melalui string log yang diparse. Attacker mengirim string khusus yang menyebabkan server me-load kode eksternal
Kerentanan Utama	Library open source kritis di-maintain oleh sangat sedikit developer, tidak ada security review yang memadai
Dampak	Ratusan ribu server rentan dalam hitungan jam, termasuk layanan pemerintah, perbankan, game (Minecraft)

Solusi yang Harus Diterapkan Programmer:

13. Software Composition Analysis (SCA) — pantau dependensi open source secara otomatis (Snyk, OWASP Dependency-Check)
14. Otomatiskan update dependensi dengan Dependabot atau Renovate
15. Jangan pernah log input pengguna yang tidak disanitasi
16. Terapkan network egress filtering — server tidak boleh sembarangan fetch URL eksternal
17. Daftarkan sistem ke vulnerability database (NVD) dan subscribe ke security advisories
18. Terapkan Web Application Firewall (WAF) sebagai defense in depth

BAGIAN III: BIAS ALGORITMA REKRUTMEN (Latihan Kelompok)

3.1 Mengidentifikasi Potensi Bias dalam Algoritma Rekrutmen

Algoritma rekrutmen modern menggunakan machine learning untuk menyaring CV, menilai kandidat, dan bahkan melakukan wawancara awal. Namun, sistem ini rentan terhadap berbagai bentuk bias.

Peta Bias dalam Proses Rekrutmen Berbasis AI

Tahap Rekrutmen	Potensi Bias	Contoh Nyata
Screening CV Otomatis	Gender bias, nama bias	Sistem Amazon: menghukum kata terkait perempuan; studi menunjukkan nama 'Abdullah' lebih jarang dipanggil dari 'Budi' dengan CV identik
Parsing Pengalaman Kerja	Institutional bias	Gap kerja karena kehamilan/merawat keluarga dinilai negatif; nama universitas ternama diberi bobot lebih
Video Interview AI	Bias penampilan fisik, aksen	Sistem HireVue dikritik menilai gerakan wajah dan nada suara yang berkorelasi dengan etnis
Tes Kemampuan Kognitif	Cultural bias, socioeconomic bias	Soal dengan referensi budaya tertentu menguntungkan kelompok tertentu
Rekomendasi Kandidat	Homophily bias	Algoritma merekomendasikan kandidat 'mirip' karyawan sukses yang sudah ada — memperkuat homogenitas

Framework Audit Bias Rekrutmen untuk Programmer

Sebagai programmer yang membangun atau menggunakan sistem rekrutmen berbasis AI:

- 19. PEMETAAN DATA: Audit komposisi data training — berapa rasio gender, etnis, usia dalam dataset historis?
- 20. ANALISIS FITUR: Identifikasi proxy features — kode pos, nama universitas, gap kerja, nama depan dapat menjadi proxy diskriminasi
- 21. FAIRNESS METRICS: Pilih dan ukur metrik keadilan yang relevan:
 - Demographic Parity: tingkat seleksi setara antar kelompok
 - Equalized Odds: true positive dan false positive rate setara
 - Calibration: probabilitas prediksi akurat di semua kelompok

- 22. BLIND TESTING: Uji sistem dengan CV identik yang hanya berbeda nama/gender
- 23. HUMAN-IN-THE-LOOP: Keputusan akhir tetap melibatkan manusia, terutama untuk tahap kritis
- 24. TRANSPARENCY: Kandidat berhak tahu kriteria penilaian dan bisa mengajukan keberatan
- 25. AUDIT BERKALA: Lakukan review bias setiap 6 bulan atau saat ada perubahan signifikan

Regulasi yang Relevan

Di Indonesia, UU No. 13/2003 tentang Ketenagakerjaan melarang diskriminasi dalam rekrutmen. GDPR Eropa mewajibkan explainability untuk keputusan otomatis yang berdampak signifikan pada seseorang. NYC Local Law 144 (2023) mewajibkan audit bias tahunan untuk alat rekrutmen AI.

BAGIAN IV: KERANGKA PENGEMBANGAN PERANGKAT LUNAK BERETIKA

4.1 Ethical Software Development Framework (ESDF)

Kerangka berikut mengintegrasikan prinsip etika ke dalam setiap fase Software Development Life Cycle (SDLC), bukan sebagai checklist tambahan, melainkan sebagai bagian integral dari proses pengembangan.

Fase 1: Perencanaan & Analisis Kebutuhan

Pertanyaan Etika yang Harus Dijawab:

- Apakah produk ini dapat disalahgunakan? Oleh siapa? Bagaimana?
- Siapa pemangku kepentingan yang paling rentan terdampak (anak-anak, lansia, disabilitas)?
- Apakah ada risiko diskriminasi terhadap kelompok tertentu?
- Data apa yang dikumpulkan? Apakah benar-benar diperlukan?

Aktivitas:

26. Ethical Risk Assessment: identifikasi risiko etika sebelum coding dimulai
27. Stakeholder mapping termasuk kelompok yang tidak memiliki suara (marginalized groups)
28. Konsultasi dengan pakar etika atau komunitas yang terdampak jika diperlukan

Fase 2: Desain Sistem

Prinsip Desain Beretika:

- Privacy by Design: privasi bukan tambahan, tapi fondasi arsitektur
- Inclusive Design: desain untuk keragaman kemampuan, budaya, dan bahasa
- Transparency by Design: pengguna dapat memahami cara kerja sistem
- Fail-Safe Defaults: konfigurasi default harus yang paling aman dan protektif

Deliverable:

29. Data Flow Diagram yang menandai setiap titik pengumpulan dan pemrosesan data
30. Threat Model yang mencakup ancaman terhadap privasi dan penyalahgunaan
31. Dokumentasi ethical trade-offs yang disadari tim

Fase 3: Implementasi (Coding)

Standar Coding Beretika:

- Secure coding standards (OWASP Top 10, CERT Coding Standards)

- Code review wajib mencakup security dan fairness, bukan hanya fungsionalitas
- Tidak menggunakan data pengguna untuk kepentingan di luar scope yang dikomunikasikan
- Dokumentasi kode yang jelas — tidak ada 'magic numbers' atau logika tersembunyi
- Tidak menanam dark patterns: manipulasi UI yang merugikan pengguna

Fase 4: Testing & Quality Assurance

Pengujian Etika yang Wajib:

Jenis Testing	Tujuan Etika
Bias Testing	Uji apakah sistem berperilaku berbeda terhadap kelompok demografi berbeda
Penetration Testing	Temukan celah keamanan sebelum attacker menemukannya
Accessibility Testing	Pastikan dapat digunakan penyandang disabilitas (WCAG 2.1)
Privacy Testing	Verifikasi data minimal, enkripsi berfungsi, akses terkontrol
Edge Case Testing	Uji perilaku sistem pada kondisi ekstrem dan input yang tidak terduga
User Acceptance Testing	Libatkan pengguna dari kelompok yang beragam

Fase 5: Deployment & Operasi

Praktik Deployment Beretika:

- 32. Rollout bertahap (canary deployment) untuk mendeteksi masalah etika sebelum dampak penuh
- 33. Monitoring aktif untuk anomali yang mengindikasikan penyalahgunaan atau diskriminasi
- 34. Incident response plan yang mencakup pelanggaran privasi dan keamanan
- 35. Mekanisme pelaporan bagi pengguna yang merasa dirugikan sistem

Fase 6: Pemeliharaan & Dekomisioning

Tanggung Jawab Jangka Panjang:

- Audit keamanan dan fairness berkala, minimal tahunan
- Update dependensi secara rutin untuk menutup kerentanan baru
- Data retention policy — hapus data yang tidak lagi diperlukan
- Saat sistem dihentikan: pastikan semua data pengguna dihapus secara permanen dan terverifikasi

4.2 Prinsip Etika Inti untuk Programmer

Prinsip	Makna Praktis
---------	---------------

Non-maleficence	Jangan membangun sistem yang kamu tahu akan digunakan untuk merugikan orang
Beneficence	Rancang sistem yang secara aktif memberikan manfaat bagi pengguna dan masyarakat
Justice	Pastikan manfaat dan risiko terdistribusi secara adil, tidak hanya menguntungkan kelompok tertentu
Autonomy	Hormati hak pengguna untuk membuat keputusan sendiri — berikan kontrol atas data mereka
Accountability	Programmer dan organisasi bertanggung jawab atas dampak sistem yang dibangun
Transparency	Bersikap jujur tentang kemampuan, keterbatasan, dan risiko sistem kepada pengguna

Deklarasi Komitmen Developer Beretika

Sebagai pengembang perangkat lunak, saya berkomitmen untuk: memahami dampak sosial dari kode yang saya tulis; tidak membangun sistem yang secara sengaja memanipulasi, mendiskriminasi, atau merugikan; melaporkan masalah etika yang saya temukan kepada atasan atau publik jika diperlukan; terus belajar tentang etika teknologi seiring perkembangan industri.

4.3 Ringkasan & Kesimpulan

Topik	Kesimpulan Utama
Privasi vs Fungsionalitas	Privacy by Design membuktikan keduanya bisa berjalan beriringan dengan pendekatan yang tepat
Bias Algoritma	Bias tidak bisa sepenuhnya dihilangkan, tetapi dapat diminimalkan dengan data yang representatif, audit, dan tim yang beragam
Pelanggaran Keamanan	Sebagian besar pelanggaran dapat dicegah dengan secure coding, enkripsi yang tepat, dan monitoring aktif
Bias Rekrutmen	Audit bias, fairness metrics, dan human-in-the-loop adalah kunci rekrutmen AI yang adil
Kerangka Etika	Integrasikan etika ke setiap fase SDLC, bukan sebagai checklist di akhir proyek

— Akhir Dokumen —